

Algoritmi e Strutture Dati – Parte B - 04/09/2020**Esercizio B1, punti ≥ 8**

Siete responsabile di un ristorante che propone menù a prezzo fisso. Avete un insieme di n piatti; il costo del piatto i -esimo è contenuto nel vettore A nella posizione $A[i]$. Il costo totale che volete proporre al pubblico è pari a k . Volete contare quanti menù diversi possono dare origine a quel costo, ottenuto sommando i costi dei singoli piatti (selezionati al massimo una volta per menù)

Scrivere un algoritmo

```
int countMenu(int[]  $A$ , int  $n$ , int  $k$ )
```

che prenda in input un vettore A contenente n interi positivi e un intero positivo k , e restituisca il numero di modi diversi per cui è possibile ottenere il valore k come somma di un sottoinsieme di elementi di A .

Discutere correttezza e complessità computazionale dell'algoritmo proposto. Specificate cosa succede nel caso più piatti abbiano lo stesso costo.

Ad esempio, se $A = [1, 3, 5, 7, 8, 4]$ e $k = 8$, è possibile ottenere tale valore come $1 + 3 + 4$, $1 + 7$, $3 + 5$, 8 . L'algoritmo dovrà restituire 4.

Esercizio B2, punti ≥ 11

Scrivere un algoritmo

```
int LCSSubstring(ITEM[]  $T$ , ITEM[]  $U$ , int  $n$ , int  $m$ )
```

che prenda in input due stringhe T , U di lunghezza n e m e restituisca la lunghezza della più lunga sottostringa comune ad entrambe. Notate che questo problema è diverso da LCS, che è basato su sottosequenze non necessariamente contigue. In questo problema le sottostringhe devono essere contigue.

Per esempio, la più lunga sottostringa comune contenuta nelle stringhe $T = \text{"abcdef"}$ e $U = \text{"abxcdexf"}$ è "cde" , e l'algoritmo dovrà restituire 3.

Discutere correttezza e complessità computazionale dell'algoritmo proposto.

Esercizio B3, punti ≥ 11

Scrivere un algoritmo

```
int printIncreasing(int[]  $A$ , int  $n$ )
```

che prenda in input un vettore A e stampi tutte le sottosequenze crescenti del vettore A .

Ad esempio, $A = [1, 3, 6, 4, 5]$ dà origine a questo output, non necessariamente in quest'ordine:

```
[1, 3, 4, 5], [3, 4, 5], [1, 4, 5], [4, 5], [1, 3, 5], [3, 5], [1, 5], [5], [1, 3, 4], [3, 4], [1, 4], [4], [1, 3, 6], [3, 6], [1, 6], [6], [1, 3], [3], [1], []
```

Discutere correttezza e complessità computazionale dell'algoritmo proposto.